

| מיקוד                                                     | משימת ליבה  |
|-----------------------------------------------------------|-------------|
| nested traversal · count/sum/max · derive a new structure | Class Stats |

## 1. Trace the Starter

אל תריצו. עקבו אחרי total, count\_high, top\_name, top\_score בכל iteration.

```
records = [("Dana", 82), ("Ali", 91), ("Maya", 76), ("Omar", 88)]
total = 0
count_high = 0
top_name = records[0][0]
top_score = records[0][1]
```

## 2. שנו record

הוסיפו record חדש עם score 95. חזו מה ישתנה ב-top\_name, count\_high, total ו-top\_score.

## 3. מבנה נגזר

בנו List בשם selected שמכיל רק names של תלמידים עם score לפחות 85. השתמשו בלולאה ו-append, לא comprehension.

## 4. Search

קלטו name\_to\_find ועברו על records כדי למצוא את ה-score המתאים. השתמשו found או תוצאה אחרת שאפשר לעקוב אחריה.

## 5. Stats

באותה לולאה חשבו total, count\_high ו-record top. סמנו איזה state variable שייך לכל pattern.

## 6. משימת ליבה מוגנת: Class Stats

בנו פתרון מלא על List of Tuples.

- manual total
- count >= 80
- top\_name + top\_score
- derived selected list for score >= 85
- friendly output + explanation

## 7. שמרו

שמרו בשם G9\_U1\_M5\_ClassStats\_<Name>.py

### בדיקת סיום ✓

```
records = [("Dana", 82), ("Ali", 91), ("Maya", 76)]
count = 0
for record in records:
    if record[1] >= 80:
        count += 1
print(count)
```

מה יודפס?

A: 1    B: 2    C: 3